

AWS GOLDEN JACKET REPORT

Project: Serverless High-Fidelity Observability & CRUD

1. Executive Summary

This project engineered a production-grade, serverless backend using API Gateway, AWS Lambda, and Amazon DynamoDB, deployed via AWS SAM. The core focus was on **High-Fidelity Structured JSON Logging** and proactive Failure Engineering, proving the ability to monitor, trace, and debug complex serverless states using advanced CloudWatch Logs Insights.

2. Business Problem

Traditional infrastructure logs fail to capture internal application state boundaries (e.g., DB connection instantiation vs. DB query execution). This opacity leads to prolonged MTTR (Mean Time To Recovery). The business required a backend that explicitly tracks lifecycle states and can be queried instantly for real-time SLA metrics without modifying code.

3. Architecture Overview

A synchronous REST API layer routing requests to a single-purpose Lambda handler. The Lambda function interfaces with an On-Demand DynamoDB table and outputs tightly structured JSON logs to CloudWatch, enabling dynamic metric calculation via Insights.

4. Architecture Diagram Explanation

- **Client -> API Gateway (REST) -> Lambda -> DynamoDB -> CloudWatch Logs**
- API Gateway acts as the front door, providing schema validation (preventing malformed payloads from wasting compute).
- Lambda executes business logic with the SDK client initialized globally for TCP connection reuse.
- DynamoDB provides single-digit millisecond persistence.

5. AWS Services Used

AWS SAM, Amazon API Gateway, AWS Lambda, Amazon DynamoDB, Amazon CloudWatch (Logs & Insights), AWS IAM.

6. Service Selection Rationale

- **API Gateway (REST):** Chosen over HTTP API for native request schema validation and advanced WAF integrations.
- **DynamoDB (On-Demand):** Selected for unpredictable, spiky workloads, eliminating the need for complex auto-scaling policies.
- **CloudWatch Logs Insights:** Selected to aggregate structured JSON without needing a 3rd party tool like Splunk or Datadog.

7. Alternative Designs

Alternative A: EC2 + RDS + CloudWatch Agent. Rejected due to high operational overhead, idle costs, and slow scaling.

Alternative B: HTTP API + Provisioned DynamoDB. Rejected because provisioned capacity requires accurate traffic forecasting, risking throttling during cold spikes.

8. Infrastructure as Code Summary

Deployed exclusively via AWS SAM (`template.yaml`). The template strictly enforces IAM least privilege, avoiding managed full-access policies in favor of granular `dynamodb:PutItem`, `GetItem`, etc.

9. Deployment Steps

1. `sam build` -> 2. `sam deploy --guided` -> 3. Inject load -> 4. Execute CW Insights queries.

10. Validation Checklist

- [x] REST API successfully routes CRUD ops.
- [x] Lambda execution role restricted to specific Table ARN.
- [x] Logs emitted as pure JSON format.

- [x] API Gateway validates payloads (400) before Lambda invocation.

11. Failure Scenarios Tested

Injected 15+ failure modes including: Missing fields (400), Type mismatch (400), Ghost item updates (409 ConditionalCheckFailed), Rapid-fire retry storms, Unsupported HTTP methods (405), and IAM permission denial (simulating misconfigurations).

12. Recovery Procedures

Identified root causes via CloudWatch Insights. For IAM failures, the fix involved reverting the SAM template IAM Role restrictions and re-deploying. For client payload errors, API Gateway blocked them automatically (auto-recovery of compute resources).

13. Pressure Test Results

The system handled rapid retry storms (10x ghost DELETES) elegantly. API Gateway deflected malformed payloads in <10ms, while Lambda cold starts averaged ~200ms and warm invocations <30ms.

14. Cost Estimate

Highly optimized. Zero idle costs. On-Demand DynamoDB costs ~\$1.25 per million writes. API Gateway REST costs ~\$3.50 per million calls. Logging costs minimized by avoiding verbose text in favor of dense JSON keys.

15. Scaling Strategy

API Gateway handles up to 10,000 RPS natively. Lambda scales out concurrently. DynamoDB On-Demand instantly accommodates double the previous peak traffic within 30 minutes.

16-18. Security, IAM & Networking Review

IAM: Implemented strict Principle of Least Privilege. No wildcard `*` resources for DB actions.

Networking: Standard AWS backbone (no VPC). VPC was deliberately omitted to prevent ENI cold-start latency since RDS/ElastiCache were not used.

19-20. Monitoring, Observability & Logging Strategy

Replaced standard `console.log()` with Structured JSON (e.g., `{"event": "DB_OPERATION_SUCCESS", "statusCode": 200}`). Built native dashboards calculating dynamic error rates directly in CloudWatch Logs Insights.

21. AWS Well-Architected Review

- **Operational Excellence:** High (Insights queries map exact SLA states).
- **Security:** High (Explicit inline IAM policies).
- **Reliability:** High (DynamoDB handles state seamlessly).
- **Performance:** High (Global SDK init connection reuse).
- **Cost Optimization:** High (Serverless pay-per-value).

22-25. Production Lessons & Traps

Hidden AWS Gotchas: The `extend` keyword from KQL does not work in CloudWatch Insights! You must perform math directly in the `stats` block (e.g., `round(sum(statusCode >= 400) / count(*) * 100, 2)`).

Optimization Opportunities: Move payload validation to API Gateway models to save Lambda invocation costs (implemented). Implement keep-alive for Node.js SDK connections (implemented via global scope).

27. Certification Mapping (SAA / DVA / SOA)

Covered Domains:

- **DVA-C02:** Troubleshooting Lambda, SDK initialization limits, API Gateway validation.
- **SOA-C02:** CloudWatch metric math, Insights query structure, IAM troubleshooting.
- **SAA-C03:** Choosing DynamoDB capacity modes, decoupling tiers.

28-31. Golden Jacket Knowledge Repository

Interview Questions

Q: How do you calculate dynamic error rates without custom metrics?

A: By emitting structured JSON logs and using CloudWatch Logs Insights `stats` command to divide 4xx/5xx responses by total requests.

Flashcards

Front: How to optimize Lambda DB connections? | **Back:** Initialize SDK client globally, outside the handler function.

Observation Library

- *"CloudWatch Logs Insights evaluates math sequentially within the stats pipeline because AWS does not use Kusto's extend keyword."*
- *"API Gateway payload validation saves compute money because it rejects 400 Bad Requests before Lambda is invoked."*

32-40. Final Project Metrics

- **Key Takeaways:** True observability requires treating logs as data schemas, not just text strings.
- **Project Difficulty:** 8/10 (Advanced Failure Engineering)
- **Confidence Score:** 95%
- **Associate Certification Readiness Score:** 90%

- **Golden Jacket Progress Score:** +100 XP

LinkedIn Project Summary: *"Just engineered a production-ready serverless backend on AWS leveraging SAM. Built high-fidelity structured JSON logging and conducted active failure injection to validate automated CloudWatch Insights SLA dashboards. #AWS #Serverless #CloudEngineering"*